Fundamentos de Programación

Tema 2: PYTHON



Docente 26-27: Daniel Castro Pereiro

Apuntes originales de Efrén Arias Jordán

Dpto Electrónica e Computación

Lenguaje PYTHON

Características

- **Python** es un lenguaje de programación totalmente gratuito, incluso para propósitos empresariales. Es interpretado y por tanto interactivo.
- **Python** es totalmente de **código abierto.**
- Python fue diseñado para **Unix**, sin embargo, en la actualidad es **multiplataforma**.
- Es un lenguaje de programación multiparadigma, que permite al programador elegir entre varios estilos:
  - programación orientada a objetos,
  - programación imperativa (y modular)
  - programación funcional.
- La sintaxis de **Python** es amigable y muy sencilla y cercana al lenguaje natural.
- Es un lenguaje muy expresivo que da lugar a **programas compactos**.
- Python es uno de los lenguajes con una mayor tasa de crecimiento en estos últimos años.
- Python reúne **lo mejor de todos los lenguajes en uno sólo**, con Python se puede desarrollar juegos, aplicaciones, sitios web …
- Es utilizado en una gran variedad de ámbitos, pero ha sido el aumento de su uso aplicado en **machine learning** y **data science**, lo que ha acelerado el crecimiento de Python.
- Es un lenguaje que está creciendo en popularidad. Algunas empresas que utilizan Python son **Yahoo**, **Google**, **Disney**, la **NASA**, **Red Hat**, etc.

Python Shell

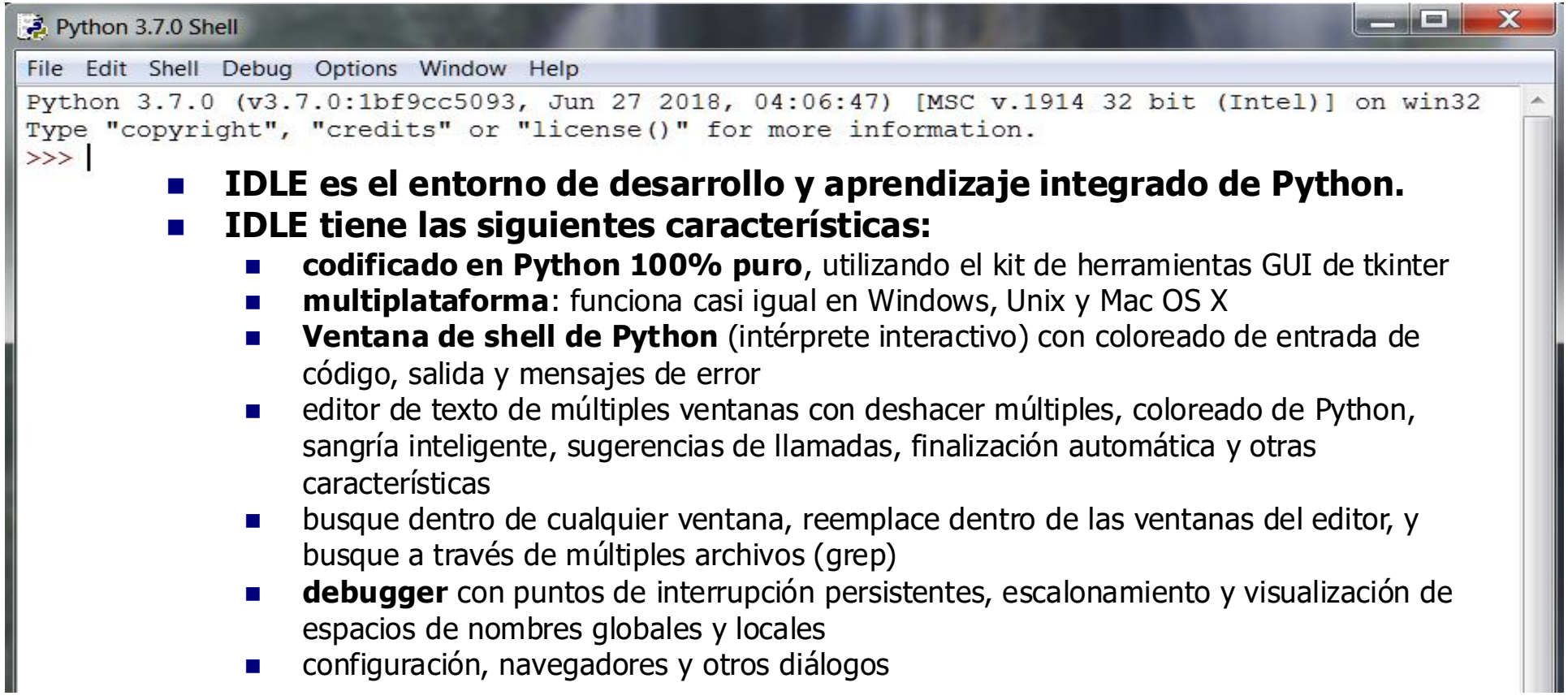
Python permite **dos modos de interacción** con el lenguaje:

- Escribir comandos directamente en el intérprete.

```
>>> "Ejemplo ejecutado en el intérprete"
'Ejemplo ejecutado en el intérprete'
```

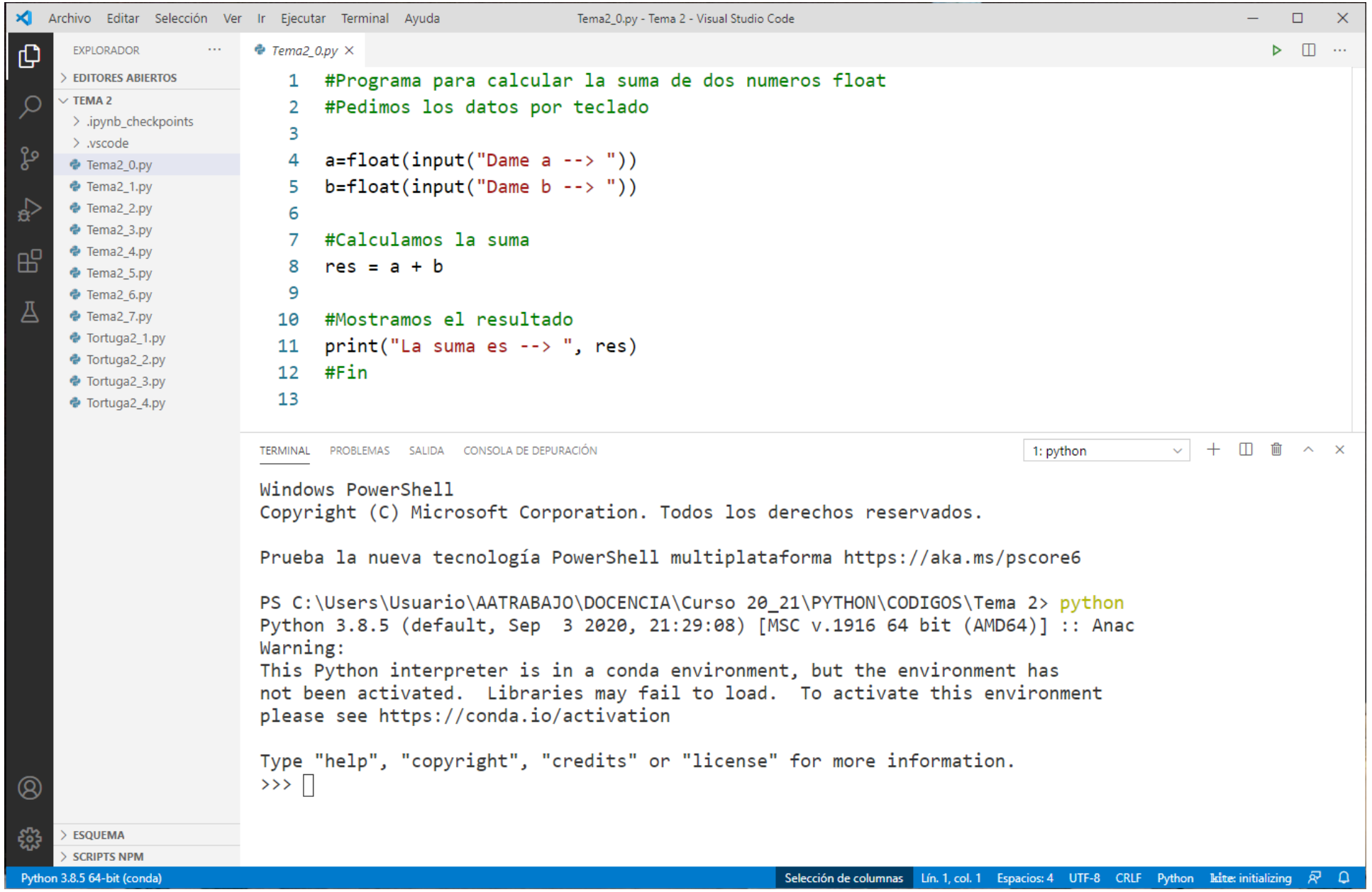
- Escribir un programa completo en un editor de textos, guardar el programa en un fichero y después usar el intérprete para ejecutarlo.

```
1 # Ejemplo para ser escrito en el editor
2 print("Ejemplo")
```

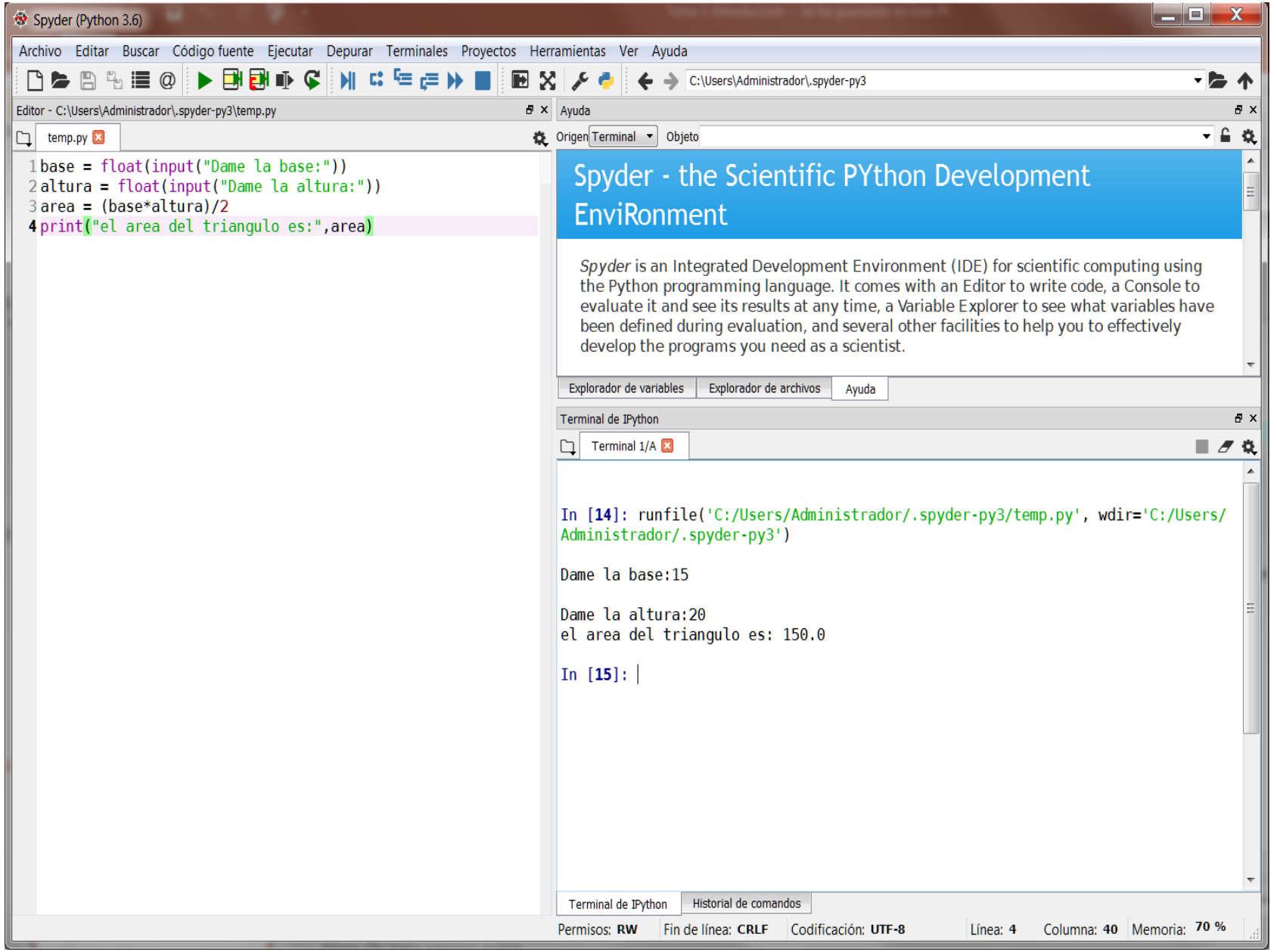


- **IDLE es el entorno de desarrollo y aprendizaje integrado de Python.**
- **IDLE tiene las siguientes características:**
  - **codificado en Python 100% puro**, utilizando el kit de herramientas GUI de tkinter
  - **multiplataforma**: funciona casi igual en Windows, Unix y Mac OS X
  - **Ventana de shell de Python** (intérprete interactivo) con coloreado de entrada de código, salida y mensajes de error
  - editor de texto de múltiples ventanas con deshacer múltiples, coloreado de Python, sangría inteligente, sugerencias de llamadas, finalización automática y otras características
  - busque dentro de cualquier ventana, reemplace dentro de las ventanas del editor, y busque a través de múltiples archivos (grep)
  - **debugger** con puntos de interrupción persistentes, escalonamiento y visualización de espacios de nombres globales y locales
  - configuración, navegadores y otros diálogos

Visual Studio Code



Spyder


Spyder (Python 3.6)
Archivo Editar Buscar Código fuente Ejecutar Depurar Terminales Proyectos Herramientas Ver Ayuda
C:\Users\Administrador\.spyder-py3
Editor - C:\Users\Administrador\.spyder-py3\temp.py
temp.py
1 base = float(input("Dame la base:"))
2 altura = float(input("Dame la altura:"))
3 area = (base*altura)/2
4 print("el area del triangulo es:",area)
Ayuda
Origen Terminal Objeto
Spyder - the Scientific PYthon Development EnviRonment
Spyder is an Integrated Development Environment (IDE) for scientific computing using the Python programming language. It comes with an Editor to write code, a Console to evaluate it and see its results at any time, a Variable Explorer to see what variables have been defined during evaluation, and several other facilities to help you to effectively develop the programs you need as a scientist.
Explorador de variables
Explorador de archivos
Ayuda
Terminal de IPython
Terminal 1/A
In [14]: runfile('C:/Users/Administrador/.spyder-py3/temp.py', wdir='C:/Users/Administrador/.spyder-py3')
Dame la base:15
Dame la altura:20
el area del triangulo es: 150.0
In [15]:
Terminal de IPython
Historial de comandos
Permisos: RW
Fin de línea: CRLF
Codificación: UTF-8
Línea: 4
Columna: 40
Memoria: 70 %

Una calculadora avanzada

Sesiones interactivas

- Los lenguajes interpretados suelen ofrecer una herramienta de ejecución interactiva. Con ella es posible dar órdenes directamente al intérprete y obtener una respuesta inmediata para cada una de ellas.
- El entorno interactivo es de gran ayuda para experimentar con fragmentos de programa antes de incluirlos en una versión definitiva.
- El **prompt** **>>>** indica que el intérprete de Python espera que nosotros introduzcamos una orden utilizando el teclado.

```
>>> 2+2
4
>>>
```

- Si deseamos acabar la sesión interactiva y salir del intérprete Python, debemos introducir una marca de final de fichero: exit()
  - CTRL d, en Unix
  - CTRL c, en Windows

Los operadores aritméticos

- Las operaciones de **suma** y **resta**, por ejemplo, se denotan con los símbolos u operadores **+** y **-** , respectivamente, y operan sobre dos valores numéricos (los operandos).

```
>>> 1+2
3
>>> 1+2+3
6
```

```
>>> 1-2+3
2
>>> 1-(2+3)
-4
```

- Los operadores de suma y resta son binarios, (**dos operandos**). El símbolo - también se usa como un operador unario (un **único operando**), que devuelve el valor de su operando cambiado de signo.

Una calculadora avanzada

- Los operadores de **multiplicación** y **división** son, respectivamente, ***** y **/** :

```
>>> 2*3
6
>>> 3/2
1.5
>>> 4/2
2.0
```

```
>>> 3*4/2
6.0
>>> 12/3*2
8.0
```

- Hay otro operador de división que es el operador de **división entera //** (sin espacio alguno entre las barras).

```
>>> 3//2
1
>>> 4//2
2
```

```
>>> -3//2
-2 (¡¡OJO con estos resultados!!)
```

- Sigamos estudiando el orden de evaluación cuando una expresión contiene diferentes operadores.

```
>>> 2*4+5
13
```

```
>>> 2+4*5
22
```

Precedencia de operadores.

- El cambio de signo tiene mayor prioridad que la multiplicación y la división:

```
>>> -2*2
-4
```

- El **operador módulo %** devuelve el resto de la división entera entre dos operandos. También es asociativo por la izquierda y su prioridad es la misma que la de la multiplicación o la división.

```
>>> 27%5
2
```

```
>>> 25%5
0
```

- El último operador que vamos a estudiar es la **exponenciación**, que se denota con dos asteriscos ****** juntos, no separados por ningún espacio en blanco:

```
>>> 2**3
8
```

```
>>> 2**3**2 #es equivalente a 2**9
512
```

Una calculadora avanzada

- La exponenciación es asociativa por la derecha y tiene mayor precedencia que cualquiera de los operadores presentados.

Operación	Operador	Aridad	Asociatividad	Precedencia
Exponenciación	**	Binario	Por la derecha	1
Identidad	+	Unario	—	2
Cambio de signo	-	Unario	—	2
Multiplicación	*	Binario	Por la izquierda	3
División	/	Binario	Por la izquierda	3
División entera	//	Binario	Por la izquierda	3
Módulo (o resto)	%	Binario	Por la izquierda	3
Suma	+	Binario	Por la izquierda	4
Resta	-	Binario	Por la izquierda	4

Símbolo	Significado	Ejemplo	Resultado
+	Suma	a = 10 + 5	a es 15
-	Resta	a = 12 – 7	a es 5
-	Negación	a = -5	a es -5
*	Multiplicación	a = 7 * 5	a es 35
**	Exponente	a = 2 ** 3	a es 8
/	División	a = 12.5 / 2	a es 6.25
//	División entera	a = 12.5 // 2	a es 6.0
%	módulo	a = 27 % 4	a es 3

Una calculadora avanzada

Errores de tecleo y excepciones

- Cuando introducimos una expresión y damos la orden de evaluarla, es posible que nos equivoquemos: Python nos lo indicará con un mensaje de error.
- El mensaje de error proporciona información acerca del tipo de error cometido y del lugar en el que este ha sido detectado.

```
>>>1+2)
File "<ipython-input-1-22dd44d0a665>", line 1
  1+2)
     ^
SyntaxError: unmatched ')'
```

- En Python los errores se denominan **excepciones**. Cuando Python es incapaz de analizar una expresión, produce una excepción.
- Cuando el intérprete interactivo detecta la excepción, nos muestra por pantalla un mensaje de error.

```
>>>1/0
Traceback (most recent call last):

  File "<ipython-input-2-9e1622b385b6>", line 1, in
<module>
    1/0
    ~^~
ZeroDivisionError: division by zero
```

- El último error es de naturaleza distinta: se trata de un error de división por cero

Una calculadora avanzada

Valores

- Desde el punto de vista de la información, los programas manejan típicamente los datos del siguiente modo:
  1. **Reciben datos** de entrada por parte de los usuarios
  2. **Procesan** esos **datos**, haciendo cálculos y operaciones con ellos
  3. **Producen resultados** que son útiles para los usuarios.

Tres tipos de datos básicos

- **Números**: 7, 3.141592
- **Textos**: "Juan", "María"
- **Valores Lógicos**: True, False

Tipos

- Cada dato o valor que se maneja en un programa es de un **tipo**. Python detecta de qué tipo es cada dato por su sintaxis.
- Los tipos definen conjuntos de valores que los programas pueden manejar y las operaciones que se pueden hacer con ellos.
  - 7 es de tipo **`int`** (de **integer**), es un número entero
  - 3.1416 es de tipo **`float`**, un número en coma flotante (**real**)
  - True es de tipo **`bool`** (de **booleano**), un valor de verdad
  - "Juan", es de tipo **`str`** (de **string**), es una cadena de caracteres
- Para saber el tipo de un valor, se puede utilizar la función **`type`**. Simplemente hay que indicar el valor del que se quiere conocer su tipo:

`>>> type(7)` `<class 'int'>`	`>>> type("Juan")` `<class 'str'>`
`>>> type(3.1416)` `<class 'float'>`	`>>> type(True)` `<class 'bool'>`

Una calculadora avanzada

- Todas las operaciones que se hacen con operandos siempre producen un resultado. Ese resultado será un valor de un cierto tipo.
- El programador debe conocer siempre el tipo que producirá una expresión sabiendo el tipo de sus operandos y la operación que realiza el operador.

Tipos entero y flotante

- Hasta el momento hemos utilizado fundamentalmente datos de tipo entero, es decir, sin decimales. Solo ocasionalmente hemos visto datos de tipo flotante, normalmente como resultado de trabajar con el operador de división convencional.
- La suma, por ejemplo, produce un resultado de tipo entero si sus dos operandos son de tipo entero, pero produce un valor de tipo flotante si uno cualquiera de sus operandos es de tipo flotante:

```
>>>1+2
3
```

```
>>>1.0+2
3.0
```

- Python sigue una regla sencilla: *si hay datos de tipos distintos, el resultado es del tipo «más general»*. Los flotantes son de tipo «más general» que los enteros.

```
>>>1+2+3+4+5+6+0.5
21.5
```

- Hemos de precisar algo respecto a la denominación de los números con decimales: el término «**reales**» no es adecuado, ya que induce a pensar en los números reales de las matemáticas.

Una calculadora avanzada

- Un número flotante debe especificarse siguiendo ciertas reglas. En principio, consta de dos partes: **mantisa** y **exponente**. El exponente, que debe ser entero, se separa de la mantisa con la letra **«e»** (o **«E»**).
- Por ejemplo, **2e3** representa el valor $2 \cdot 10^3$ y **3.2E-3** el valor $3.2 \cdot 10^{-3}$

El tipo de datos booleano (y sus operadores)

- Hay otro tipo de datos que se usa muy frecuentemente en programación, el tipo de datos lógico o **booleano**.
- Un dato de tipo lógico solo puede presentar uno de dos valores: **`True`** o **`False`**, es decir, verdadero o falso.

- Hay tres operadores lógicos en Python: la «**y lógica**» o conjunción (**`and`**), la «**o lógica**» o disyunción (**`or`**) y el «**no lógico**» o negación (**`not`**).

```
False   or    False   =   False
False   or    True    =   True
True    or    False   =   True
True    or    True    =   True
```

```
False   and   False   =   False
False   and   True    =   False
True    and   False   =   False
True    and   True    =   True
```

```
not     False     =     True
not     True      =     False
```

Una calculadora avanzada

```
>>>True and False
False
>>>not True
False
```

```
>>>(True and False)or
True
True
```

Operación	Operador	Aridad	Asociatividad	Precedencia
Negación	not	Unario	—	alta
Conjunción	and	Binario	Por la izquierda	media
Disyunción	or	Binario	Por la izquierda	baja

- Hay una familia de operadores que devuelven valores booleanos. Entre ellos tenemos a los operadores de comparación (**relacionales**)
- Uno de ellos es el **operador de igualdad**, que devuelve **`True`** si los valores comparados son iguales **`False`** en caso contrario

```
>>>2==1+1
True
>>>2.1==2.1
True
```

Operadores

- **Relacionales:**

```
== (igualdad)
!= (distinto)
< (menor que)
<= (menor o igual que)
> (mayor que)
>= (mayor o igual que)
```

- Los operadores relacionales sirven para determinar si es cierta o no una relación de orden entre dos valores, o si éstos son iguales o distintos.
  - Por ejemplo, podemos comprobar si 3 <= 5.

Símbolo	Significado	Ejemplo	Resultado
==	Igual que	5 == 7	False
!=	Distinto que	'rojo'!='verde'	True
<	Menor que	8 < 12	True
>	Mayor que	1 > 7	False
<=	Menor o igual que	12 <= 12	True
>=	Mayor o igual que	4 >= 5	False

Una calculadora avanzada

- Por último, se pueden hacer las típicas operaciones lógicas "**y**", "**o**" y "**no**". De nuevo en este caso el valor que se produce es de tipo **bool** ya que la operación podrá ser cierta o falsa.

- **Lógicos**:

```
and (y-lógico)
or (o-lógico)
not (no-lógico)
```

Operador	Ejemplo	Resultado	
and (y)	5 == 7 and 7 > 12	0 y 0	False
	9 < 12 and 12 > 7	1 y 1	True
	9 < 12 and 12 > 15	1 y 0	False
or (o)	12 == 12 or 15 < 7	1 o 0	True
	7 > 5 or 9 < 12	1 o 1	True
	1 == 2 or 5 > 7	0 o 0	False

Otros tipos de datos numéricos

- Un **número complejo** puro finaliza siempre con la letra **j**, que representa el valor $\sqrt{-1}$

- Un número complejo con parte real se expresa sumando la parte real a un complejo puro. He aquí ejemplos de números complejos: **4j, 1+2j**, **2.0-3j**, **1+0.354j**. Y, para acabar, un ejemplo de expresiones aritméticas con complejos:

```
>>> (1+3j)/(2+1j)
(1+1j)
```

```
>>> (1+2j)*(1-2j)
(5+0j)
```

- En muchos casos, antes de utilizar una función o un tipo de dato nuevo, debemos importar el módulo que los contiene y/o define. Para ello utilizaremos la expresión

```
>>>from <módulo> import <función o dato>
```

Una calculadora avanzada

- Los números de tipo **`decimal`** dan una solución al problema de la imprecisión de los flotantes. Esta imprecisión es inaceptable cuando manejamos, por ejemplo, dinero. El tipo **Decimal** maneja decimales con precisión:

```
>>> from decimal import Decimal
>>> a=Decimal('0.1')
>>> b=Decimal('0.3')
>>> a-b
Decimal('-0.2')
```

- Otro modo de abordar el problema de la imprecisión de los flotantes es usar el tipo **`Fraction`**, que permite manipular números formados por un numerador y un denominador:

```
>>> from fractions import Fraction
>>> Fraction(2,4)
Fraction(1, 2)
>>> Fraction(1,10)-Fraction(3,10)
Fraction(-1, 5)
```

Uso del intérprete de Python como calculadora.

```
>>> 2+3 # suma
5
>>> 2 + 3 # suma
5
>>> 5 - 7 # resta
-2
>>> 5 * 3 # multiplicación
15
>>> 8 // 3 # división entera
2
>>> 8.0 / 3 # división en punto flotante
2.6666666666666665
>>> 8.0 // 3 # división entera
2.0
>>> 3 ** 2 # exponenciación
9
>>> 11 % 4 # módulo (resto de la división entera)
3
>>> 5 - 2 * 3 # operaciones encadenadas
-1
>>> (5 - 2) * 3 # con los paréntesis se cambia la
                  precedencia de los operadores
9
```

Una calculadora avanzada

El tipo de datos cadena

- Una **cadena** es una secuencia de caracteres (letras, números, espacios, marcas de puntuación, etc.) y en Python se distingue porque va encerrada entre comillas simples o dobles .
- Las cadenas pueden usarse para representar información textual: nombres de personas, nombres de colores, matrículas de coche... Las cadenas también pueden almacenarse en variables.

```
>>>nombre = 'Pepe'
>>>nombre
'Pepe'
```

- Es posible realizar operaciones con cadenas. Por ejemplo, podemos sumar (realmente es **concatenar**) cadenas añadiendo una a otra:

```
>>>'a'+'b'
'ab'
```

```
>>>nombre = 'Pepe'
>>>nombre + ' '+'Cano'
'Pepe Cano'
```

- Hay un operador de repetición de cadenas, que necesita dos datos: uno de tipo cadena y otro de tipo entero. El resultado es la concatenación de la cadena consigo misma tantas veces como indique el número entero:

```
>>>'Hola '*5
'Hola Hola Hola Hola Hola '
>>>'-'*40
'----------------------------------------'
```

- Una cadena no es un identificador

```
>>>a=1
>>> 'a'
'a'
>>> a
1
```

Una calculadora avanzada

- Intuitivamente es natural pensar que una relación de orden o igualdad puede ser cierta o no, es decir, el resultado de esa operación será un valor lógico siempre, independientemente de si los valores son números o cadenas de caracteres.
- Los valores de tipo **`str`** también se pueden comparar, el valor que se produce depende de su ordenación alfabética, distinguiéndose entre mayúsculas y minúsculas.

Ejemplos

```
>>> "Antonio" < "Juan"
True
>>> 7 + 4
11
>>> 3.1416 * 2
6.2832
>>> "Juan" + " Alonso”
'Juan Alonso'
```

```
>>> True and False
False
>>> 2 / 3                  #división real
0.6666666666666663
>>> 2 // 3                 #división entera
0
>>> 2.0 // 3    #división entera
0
>>> 2 % 3                  #resto div entera
2
>>> 2 ** 3                 # potencia
8
>>>> "Pe" * 4
'PePePePe'
>>>> "Anti" + "natural"
'Antinatural'
>>> not True
False
>>> 2 > 3
False
>>> 3 <= 3.1416
True
>>> 2 == 3
False
>>> 2 != 3
True
```

Variables, expresiones, asignación

Variables

- En ocasiones deseamos que el ordenador recuerde ciertos valores para usarlos más adelante.

```
>>>pi=3.141592
>>>r=1.213
>>>pi*r**2
4.622441079448
```

- Una **variable** es el nombre que se utiliza en un programa para representar un dato o valor que puede cambiar y, por tanto, será de un tipo. Ese valor además se guardará en una **posición de la memoria**.
- El acto de dar valor a una variable se denomina **asignación**. Al asignar un valor a una variable que no existía, Python reserva un espacio en la memoria, almacena el valor en él y crea una asociación entre el nombre de la variable y su dirección de memoria.
- La posición de la memoria donde se encuentra el valor que representa una variable es intrascendente para la ejecución del programa; en ejecuciones distintas, el valor se situará en posiciones de la memoria diferentes.
- Supongamos que necesitamos hacer un programa que sirva para calcular el área de un rectángulo, dados los valores de su base y su altura. Sabemos que el cálculo del área de un rectángulo se realiza mediante la ecuación:

```
area = base*altura
```

```
>>> base = 5
>>> altura = 4
>>> base * altura
20
>>> altura = 2
>>> base * altura
10
```

Variables, expresiones, asignación

Otras consideraciones sobre las variables

- Hay dos aspectos importantes a la hora de trabajar con las variables que tendrán nuestros programas.
- La primera es **elegir un nombre adecuado** para cada variable del programa y la segunda **seleccionar qué variables** necesitamos usar.
- El nombre de una variable puede estar formado por letras (**a, b, c,. . .**), dígitos (**0,1,2,. . .**) y el carácter guion bajo o subrayado (_). Además, deben tenerse en cuenta las siguientes restricciones:
  - el nombre no puede empezar por un dígito
  - el nombre no puede ser igual que una palabra reservada del lenguaje
  - mayúsculas y minúsculas son diferentes

- Las palabras reservadas son 35 en Python 3.12:

`False`	`None`	`True`	`and`	`as`
`assert`	`async`	`await`	`break`	`class`
`continue`	`def`	`del`	`elif`	`else`
`except`	`finally`	`for`	`from`	`global`
`if`	`import`	`in`	`is`	`lambda`
`nonlocal`	`not`	`or`	`pass`	`raise`
`return`	`try`	`while`	`with`	`yield`

- Se recomienda utilizar **nombres lo más descriptivos posibles**, aunque sean largos y formados por varias palabras

```
nombre, velocidad_final, interes,
codigo_postal
```

- Python distingue entre mayúsculas y minúsculas, así que **area**, **Area** y **AREA** son tres identificadores válidos y diferentes
- Si en un programa deseemos guardar alguna información para luego acceder a ella, entonces es necesario crear una variable

Variables, expresiones, asignación

Asignación

- Una de las instrucciones más importantes para la construcción de programas es la **asignación**. Es la instrucción que nos permite cambiar el valor de una variable.

```
Variable=expresión
```

- El efecto que tiene una asignación es diferente en función de cómo sea la expresión que aparece en la parte derecha de la asignación. Hay dos opciones:
  1. si se asigna un valor o el resultado de una operación, entonces se reserva un nuevo espacio en la memoria para contenerlo
  2. si se asigna otra variable, entonces las dos variables contienen el mismo valor

```
>>> base = 5
>>> altura = base
>>> area = base * altura
>>> area
25
```

Expresiones

- Una **expresión** es una combinación de operadores y operandos (valores, variables) que produce un resultado (valor) de un cierto tipo.
- La mayor dificultad es la precedencia de los operadores. Cuatro reglas más básicas:
  - lo que está entre paréntesis se hace primero.
  - los operadores aritméticos tienen más precedencia que los relacionales, y los relacionales más que los lógicos.
  - los operadores unarios de un grupo más que los binarios de ese mismo grupo.
  - ** más que los multiplicativos **(*, /, %)**, y éstos más que los aditivos **(+, -)**. Esta regla también se da con los lógicos (**and** más precedencia que **or**).

Variables, expresiones, asignación

Operación	Operador	Aridad	Asociatividad	Precedencia
Exponenciación	**	Binario	Por la derecha	1
Identidad	+	Unario	—	2
Cambio de signo	-	Unario	—	2
Multiplicación	*	Binario	Por la izquierda	3
División	/	Binario	Por la izquierda	3
División entera	//	Binario	Por la izquierda	3
Módulo (o resto)	%	Binario	Por la izquierda	3
Suma	+	Binario	Por la izquierda	4
Resta	-	Binario	Por la izquierda	4
Igual que	==	Binario	—	5
Distinto de	!=	Binario	—	5
Menor que	<	Binario	—	5
Menor o igual que	<=	Binario	—	5
Mayor que	>	Binario	—	5
Mayor o Igual que	>=	Binario	—	5
Negación	`not`	Unario	—	6
Conjunción	`and`	Binario	Por la izquierda	7
Disyunción	`or`	Binario	Por la izquierda	8

Variables, expresiones, asignación

Asignaciones con operador

- La sentencia **`i=i+1`** aplica un incremento unitario al contenido de la variable i. Incrementar el valor de una variable en una cantidad cualquiera es tan frecuente que existe una forma compacta en Python.
- El incremento de i puede denotarse así: **`i+=1`** (sin espacio alguno entre el + y el =). Se puede incrementar una variable con cualquier cantidad, incluso con una que resulte de evaluar una expresión

```
>>>a=3
>>>a+=4
>>>a
7
```

- Estas formas compactas no aportan nada nuevo … salvo comodidad. Todos los operadores aritméticos tienen su asignación con operador asociada.

```
>>> z=1
>>> z+=2
>>> z*=2
>>> z//=2
>>> z-=2
>>> z%=2
>>> z**=2
>>> z/=2
>>> z
0.5
```

Variables no inicializadas

- En Python, la primera operación sobre una variable debe ser la asignación de un valor.
- La asignación de un valor inicial a una variable se denomina **inicialización** de la variable. No se puede usar una variable a la que no se ha asignado previamente un valor.

```
>>>x+2
Traceback (most recent call last):
  File "<ipython-input-30-56c530d9a4bc>", line 1, in
<module>
    x+2
    ^
NameError: name 'x' is not defined
```

Variables, expresiones, asignación

Literales de entero

- Normalmente trabajamos con números en base 10. En informática es muy común trabajar con base 2, base 8 y base 16. Python permite expresar números en estas tres bases.
- Los números en base 2 se expresan con el prefijo **0b** (o **0B**) seguido de una sucesión de unos y ceros.

```
>>>0b1+0b0001
2
```

- Los números en octal (base 8), se forman con **0o** (o **0O**) seguida de uno o más dígitos entre 0 y 7.

```
>>>0o10+0o10
16
```

- Finalmente, los números en hexadecimal (base 16), empiezan por **0x** (o **0X**) y siguen con uno o más dígitos entre 0 y 9 o letras minúsculas o mayúsculas entre A y F.

```
>>>0xA+0x8
18
```

Conversiones entre tipos

- A veces los programas reciben valores que necesitan convertirlos en valores de otro tipo (datos que se reciben como **`string`** pero que representan números).
- Para hacer esas conversiones:
  - primero hay que indicar el nombre del tipo al que se quiere convertir el valor
  - y posteriormente entre paréntesis el valor que se quiere convertir.

`>>> int("7")` `7`	`>>> str(3.1416)` `'3.1416'`
`>>> str(7)` `'7'`	`>>> int(3.1416)` `3`
`>>> float("3.1416")` `3.1416`	`>>> float(7)` `7.0`

Variables, expresiones, asignación

Funciones predefinidas

- Python también proporciona funciones predefinidas que podemos utilizar en las expresiones.

Algunas funciones sobre valores numéricos

- La función **`abs()`**, por ejemplo, calcula el **valor absoluto** de un número.

```
>>>abs(-3)
3
```

```
>>>abs(3)
3
```

- El número sobre el que se aplica la función se denomina **argumento** de la función y debe ir encerrado entre paréntesis:

```
>>>abs(0)
0
>>>abs 0
  File "<ipython-input-2-ee9e6f391021>", line 1
    abs 0
        ^
SyntaxError: invalid syntax
```

- **`float()`: conversión a flotante**. Si recibe un número entero como argumento, devuelve el mismo número convertido en un flotante equivalente.

```
>>> float(3)
3.0
```

```
>>> float('3.2')
3.2
```

- **`int()`: conversión a entero**. Si recibe un número flotante como argumento, devuelve el entero que se obtiene eliminando la parte fraccionaria.

```
>>> int(2.1)
2
```

```
>>> int(-2.9)
-2
```

- **`str()`: conversión a cadena**. Recibe un número y devuelve una representación de este como cadena.

```
>>> str(2.1)
'2.1'
```

```
>>> str(2.34e+10)
'23400000000.0'
```

Variables, expresiones, asignación

Dos funciones básicas para cadenas: ord y chr

- Cuando Python compara dos cadenas lo hace carácter a carácter. Cada carácter es un entero de 16 bits. La letra **a**, por ejemplo, se codifica con el entero 97, y la **b** con el entero 98 (B con 66)

```
>>> 'abajo'< 'arriba'
True
```

```
>>> 'Barco'<'ancla'
True
valor_unicode = ord('B')
print(valor_unicode)
```

- Las letras acentuadas plantean problemas similares, pues tienen valores numéricos mayores que sus versiones sin acentuar:

```
>>> 'ábaco'<'ajo'
False
```

- Para conocer el valor numérico que corresponde a un carácter, se utiliza la función predefinida **`ord()`**

```
>>> ord('a')
97
```

```
>>> ord('A')
65
```

- La función **`chr()`** hace lo contrario: devuelve un carácter, dado su valor numérico.

```
>>> chr(97)
'a'
```

```
>>> chr(225)
'á'
```

Módulos e importación de funciones y variables

- Python también proporciona funciones trigonométricas, logaritmos, etc., pero no están directamente disponibles cuando iniciamos una sesión. Antes de utilizarlas hemos de indicar a Python que vamos a hacerlo. Para ello, importamos cada función de un módulo

El módulo `math`

- Empezaremos por importar la función seno (**`sin`**, del inglés «sinus») del módulo matemático **`math.`**

```
>>> from math import sin
>>> sin(0)
0.0
```

```
>>> sin(1)
0.841470984807896
```

Variables, expresiones, asignación

- En una misma sentencia podemos importar más de una función, o bien importarlas todas con un *****

```
>>> from math import *
>>> cos(0)
1.0
```

- Python permite indicar solo el nombre del módulo al importar. De esta forma, todas las funciones del módulo **`math`** están disponibles, pero usando el nombre del módulo y un punto como prefijo:

```
>>> import math
>>> math.sin(0)
0.0
```

```
>>> math.cos(0)
1.0
```

- En el módulo matemático se definen, además, algunas constantes de interés:

```
>>> from math import pi,e
>>> pi
3.141592653589793
```

```
>>> e
2.71828182845904
```

- Algunas de las funciones que encontramos en el módulo math: (**x debe estar en radianes**)
  - **`sin(x), cos(x)`** Seno y coseno de x
  - **`tan(x)`** Tangente de x
  - **`exp(x)`** Nos da e^x
  - **`ceil(x)`** Redondeo arriba de x.
  - **`floor(x)`** Redondeo abajo de x.
  - **`log(x)`** Logaritmo neperiano de x
  - **`log10(x)`** Logaritmo decimal de x
  - **`sqrt(x)`** Raíz cuadrada de x
  - **`pow(x,n)`** Potencia x^n
- Existe un gran número de módulos (diseño de **aplicaciones web**, diseño de **interfaces de usuario**, **compresión de datos**, **criptografía**, **multimedia**, etc)

Variables, expresiones, asignación

El módulo `random`

- Permite acceder a varias funciones para la generación de **números aleatorios**.
  - **`seed()`** Inicializa la semilla aleatoria
  - **`random()`** Genera número entre 0.0 y 1.0
  - **`randint(f1,f2)`** Genera un número aleatorio entre f1 y f2
  - **`random.uniform(0.0,10.0)`** Genera float aleatorio entre 0.0 y 10.0
  - **`random.gauss(5.0,1.0)`** Genera float aleatorio media 5.0 y var 1.0
  - **`random.choice(('rojo','verde','azul'))`** Genera elemento aleatorio

```
#Tema2_1 #Calcula la hipotenusa

#Calcula la hipotenusa

import math

import random
random.seed()

c1= random.randint(1,100)

c2= random.randint(1,100)

h= math.sqrt((c1**2) + (c2**2))

print("La hipotenusa del triángulo de catetos
",c1," y ",c2," es ",h)
```

Variables, expresiones, asignación

El módulo `str`

- Contiene funciones y valores relacionados con el texto. No es necesario importar nada, pues ya está incluido desde el comienzo.
  - **`str.lower(t)` y `str.upper(t)`** convierten un texto a minúsculas y a mayúsculas, respectivamente.
  - **`str.islower(t)` y `str.isupper(t)`** devuelven **`True`** cuando **t** contiene exclusivamente caracteres en minúsculas o en mayúsculas, respectivamente.
- Algunas de las funciones que encontramos en el módulo **`str`**:
  - **`str.strip()`** Elimina espacios a izda y dcha
  - **`str.isdigit(t)`** Determina si **t** solo tiene números
  - **`str.isalpha(t)`** Determina si **t** solo tiene letras
  - **`str.isspace(t)`** Determina si **t** sólo tiene espacios
  - **`str.find(t1,t2)`** Determina la posición de **t2** en **t1**
  - **`str.format(t,e)`** Formatea **t** según **e**
- La función **`str.format(t,e)`** merece una explicación más detallada. Se trata de crear una cadena a partir de un valor **e** (en realidad, de cualquier número de valores), según las instrucciones en la cadena de formateo **t**.
- Los lugares en donde se deben colocar los valores se encierran entre llaves, con un número que indica la posición en la lista. A estos les llamaremos "**marca**".

a)

```
>>>print(str.format("{0}, {1} y {2}", 1, 2, 3))
```

b)

```
>>>print("{0}, {1} y {2}".format(1,2,3))
1, 2 y 3
>>> print(str.format("{0}x{0}={1}",3, 9))
3 x 3 = 9
>>> nombre = "Juan"
>>> edad = 30
>>> print("Hola, mi nombre es {} y tengo {}
años.".format(nombre, edad))
```

Variables, expresiones, asignación

```
>>> import math
>>>print( str.format("{0:5.2f}",math.pi))
 3.14
>>> print("{0:5.2f}" .format(pi))
>>> print(str.format("{0:5.30f}",math.pi))
3.141592653589793115997963468544
>>>print(str.format("{0:08.4f}",  math.pi))
003.1416
>>> print(str.format("{0:15s}", "Hola, mundo"))
Hola, mundo
```

- Muestra "Hola, mundo " por pantalla, es decir, con cuatro espacios "extra" a la derecha.

```
>>> print(str.format("{0:4d}", 128))
 128
>>> print(str.format("{0:4x}", 128))
  80
>>> print(str.format("{0:4o}", 128))
 200
>>> print(str.format("{0:4b}", 128))
10000000
```

- Finalmente, los indicadores de tipo **'b'** **'x'** y **'o'** indican que el valor entero (debe ser entero, obligatoriamente), debe formatearse como un número binario, hexadecimal u octal, respectivamente.

Variables, expresiones, asignación

Métodos

- Los datos de ciertos tipos permiten invocar unas funciones especiales: los denominados «**métodos**».
- Hemos visto que las funciones se invocan así:

```
función (arg1,arg2,arg3….).
```

- Los métodos son funciones especiales, pues se invocan del siguiente modo:

```
arg1.método(arg2,arg3…).
```

- Esta sintaxis recalca el hecho de que, para un método, el primer argumento es muy especial

Unos métodos sencillos para manipular cadenas…

- Un método permite, por ejemplo, obtener una versión en minúsculas de la cadena sobre la que se invoca:

```
>>> cadena="Un EJEMPLO de Cadena"
>>> cadena.lower()
'un ejemplo de cadena'
>>> "OTRO EJEMPLO".lower()
'otro ejemplo'
```

- Existe otro método, **`upper`**, que pasa todos los caracteres a mayúsculas.

```
>>> cadena="Un EJEMPLO de Cadena"
>>> cadena.upper()
'UN EJEMPLO DE CADENA’
```

- Y otro, `title`, que pasa la inicial de cada palabra a mayúsculas.

```
>>> "PEDRO FDEZ. MAS".title()
'Pedro Fdez. Mas'
```

Variables, expresiones, asignación

- Algunos métodos aceptan argumentos. El método **replace**, por ejemplo, recibe como argumento dos cadenas: **un patrón** y **un reemplazo**.
- Se busca el patrón en la cadena y sustituye todas sus apariciones por la cadena de reemplazo.

```
>>> 'un pequeño ejemplo'.replace('pequeño', 'gran')
'un gran ejemplo'
>>> una_cadena='abc'.replace('b','B')
>>> una_cadena
'aBc'
```

- El método **format()** se utiliza para mostrar la información con formato, por ejemplo, interpolar valores en una cadena, es decir, sustituir una marca especial por el valor de una expresión

```
"El numero {0} ha sido interpolado".format(1.23)
'El numero 1.23 ha sido interpolado'
```

- Podríamos hacer esto con **replace()**:

```
"El numero {0} ha sido
interpolado".replace('{0}','1.23')
'El numero 1.23 ha sido interpolado’
```

- El método **format()** presenta algunas ventajas:
  - su uso conduce a una expresión más breve y el número 1.23 se ha podido suministrar como número (**replace** exige que sea una cadena).
  - Podemos interpolar más de un valor con una sola llamada a **format**:

```
>>> "Los números {1} y {0} han sido
interpolados".format(1.23,9.999)
'Los números 9.999 y 1.23 han sido interpolados'
```

Variables, expresiones, asignación

- Las marcas pueden modificarse para controlar el aspecto de la información. Imaginemos que deseamos mostrar los valores flotantes redondeados con un solo decimal:

```
>>> "Los números {0:.1f} y {1:.1f} han sido
interpolados".format(1.23,9.999)
'Los números 1.2 y 10.0 han sido interpolados'
```

- Las marcas de formato permiten controlar con precisión el modo en el que se muestran los datos. La forma general de una marca de formato es:

```
{campo!conversión:formato}
```

- El «**campo**» es el número que identifica el número de argumento que se desea interpolar en la cadena (aunque admite otros valores). Olvidemos por el momento el fragmento «**!conversión**» y centrémonos en la parte «**formato**». Su forma general es esta:

```
[[relleno]alineamiento][signo][#][0][ancho][.precisión][código de tipo]
```

- Cada elemento entre corchetes es opcional. Veamos algunas posibilidades para cada uno de esos elementos:
  - **`relleno`**: Carácter con el que rellenar los espacios que requiere un alineamiento (por defecto, espacio en blanco).
  - **`alineamiento`**: Carácter **<** para alinear a la izquierda (por defecto para la mayoría de los tipos de datos); carácter **>** para alinear a la derecha (por defecto para números); carácter **^** para centrar en el espacio disponible.
  - **`signo:`** Carácter **+** para forzar la aparición de un signo incluso en números positivos; carácter **-** para indicar que el signo solo debe aparecer con números negativos (es el valor por defecto); un espacio en blanco para indicar que los números positivos deben ir precedidos por un espacio en blanco.

Variables, expresiones, asignación

- **`#`**: Si aparece, los enteros que se muestran en binario, octal o hexadecimal irán precedidos por **0b**, **0o** o **0x**
- **`0:`** Si aparece, se usa el carácter **0** para sustituir los espacios en blanco.
- **`ancho`**: Es un número entero que indica cuántos caracteres queremos que ocupe (como mínimo) el valor representado.
- **`.precisión`**: número de decimales con que queremos representar un número flotante.
- **`código de tipo:`** carácter que indica el tipo de representación que se desea. Es diferente según el tipo de datos del valor. He aquí algunos de sus posibles valores:

Números enteros	
Carácter	**Acción**
b	en binario.
c	como carácter Unicode.
d	en base diez (es el valor por defecto).
o	en octal.
x	en hexadecimal.
n	igual que d, pero como número adaptado a la cultura local (en español, por ejemplo, el punto decimal se muestra como una coma).
Números flotantes	
Carácter	**Acción**
e	notación exponente.
f	notación de punto fijo.
g	notación en formato general, que solo es exponente para números grandes (es el valor por defecto).
n	igual que g, pero como número adaptado a la cultura local.
%	muestra el número multiplicado por 100, en formato f y seguido de un símbolo de porcentaje.

Ejemplos

Uso del intérprete de Python como una calculadora.

```
>>> 2+3 # suma
5
>>> 2 + 3 # suma
5
>>> 5 - 7 # resta
-2
>>> 5 * 3 # multiplicación
15
>>> 8 // 3 # división entera
2
>>> 8.0 / 3 # división en punto flotante
2.6666666666666665
>>> 8.0 // 3 # división entera
2.0
>>> 3 ** 2 # exponenciación
9
>>> 11 % 4 # módulo (resto de la división entera)
3
>>> 5 - 2 * 3 # operaciones encadenadas
-1
>>> (5 - 2) * 3 # con los paréntesis se cambia la
precedencia de los operadores
9
```

Variables y tipos de datos básicos

```
>>> x = 5 # asigna el valor 5 a la variable de
nombre x
>>> print (x) # visualiza en el terminal el
valor de la variable x
5
>>> type(x) # devuelve el tipo de la variable x
<type 'int'>
>>> y = 2.3 # asignación
>>> type(y) # tipo de dato
<type 'float'>
>>> z = x + y # asignar a la variable z el valor
de la evaluación de una expresión
>>> print (z)
7.3
>>> type(z)
<type 'float'>
>>> cadena = “Hola a todos" # asigna a la
variable cadena una cadena de caracteres
>>> print (cadena)
Hola a todos
```

Ejemplos

```
>>> type(cadena) # devuelve el tipo string (cadena de caracteres)
<type 'str'>
>>> c = 4 + 2j # asigna a la variable c un numero complejo (ojo: 1+1j, no vale 1+j)
>>> print (c)
(4+2j)
>>> type(c)
<type 'complex'>
>>> b = True # asigna a la variable b un booleano
>>> print (b)
True
>>> type(b)
<type 'bool'>
>>> c2 = 3 - 4j
>>> c + c2 # suma de números complejos
(7-2j)
>>> c * c2 # multiplicación de números complejos
(20-10j)
>>> a, b, c = 5, False, “Hola" #asignaciones múltiples
>>> print (a)
5
>>> print (b)
False
>>> print (c)
Hola
>>> x = -3.4
>>> abs(x) # valor absoluto
3.4
>>> c = 4 - 3j # número complejo
>>> abs(c) # valor absoluto (módulo del complejo)
5.0
>>> type(x)
<type 'float'>
>>> y = int(x) # convierte x a un número entero
>>> y
-3
>>> type(y)
<type 'int'>
>>> float(y) # convierte y a un número real o flotante
-3.0
>>> 2>3         # compara ambos elementos (2 > 3). Devuelve False
False
```

Ejemplos

```
>>> 3>2         # devuelve True
True
>>> from math import * # importa todas las funciones
del módulo math
>>> ceil(1.001)
2
>>> floor(1.001)
1
>>> factorial(10)
3628800
>>> gcd(10,125) # MCD(10,125)
5
>>> trunc(1.001)
1
>>> round(1.999)
2
>>> pi # constante pi definida en el módulo math
3.141592653589793
>>> e # constante e definida en el módulo math
2.718281828459045
>>> sin(0)
0.0
>>> sin(pi/2)
1.0
>>> sin(pi/4)
0.7071067811865476
>>> sqrt(3) # raíz cuadrada de 3
1.7320508075688772
>>> log(e) # logaritmo neperiano
1.0
>>> exp(2) # función exponencial e^2
7.38905609893065
>>> fabs(-7.3) # valor absoluto de un número real
7.3
>>> cos(pi)
-1.0
>>> tan(pi/6)
0.5773502691896257
>>> hypot(12,5)  #hipotenusa
13.0
>>> atan(0.5773502691896257)
0.5235987755982988
>>> asin(0.7071067811865476)
0.7853981633974484
```

Ejemplos

```
>>> exp(5)
148.4131591025766
>>> e**5
148.4131591025765
>>> log(148.41315910257657)  #log_e
5.0
>>> log2(148.41315910257657) #log_2
7.213475204444817
>>> log10(148.41315910257657)#log_10
2.171472409516258
>>> log(148,3)               #log base 3
4.548658635361187
>>> log(1.0000025)
2.4999968749105643e-06
>>> log1p(0.0000025)         #log(1+x)
2.4999968750052084e-06
>>> pow(12.5, 2.8)
1178.5500657314767
>>> pow(144, 0.5)
12.0
>>> sqrt(144)
12.0
```

- Al tratar con **números complejos**, el módulo **cmath** puede ser de gran ayuda. El módulo **cmath** también nos permite usar funciones matemáticas regulares con números complejos.

```
>>> # importa todas las funciones del módulo cmath
>>> from cmath import *
>>> # forma polar
>>> polar(complex(1.0, 1.0))
(1.4142135623730951, 0.7853981633974483)
>>> phase(complex(1.0, 1.0))
0.7853981633974483
>>> abs(complex(1.0, 1.0))
1.4142135623730951

>>> sqrt(complex(25.0, 25.0))
(5.49342056733905+2.2754493028111367j)
>>> cos(complex(25.0, 25.0))
(35685729345.58163+4764987221.458499j)
```

Uso de entrada/salida por consola

Entrada por teclado

- Una característica muy importante de los programas es que sean capaces de interactuar con el usuario, es decir, que sean capaces de pedir información al usuario por teclado (entrada estándar) y de mostrar los resultados por pantalla (salida estándar).
- El siguiente programa en Pyhton calcula el área de un triángulo, pero la base y la altura siempre son las mismas y no se informa del área calculada.

```
base = 5.0
altura = 3.0
area = (base*altura)/2
```

- Para aplicarlo a cualquier triangulo que quiera el usuario:

```
base = float(input("Dame la base: "))
altura = float(input("Dame la altura: "))
area = (base*altura)/2
print(area)
```

- La forma más sencilla de obtener información por parte del usuario es mediante la función **`input`**. Esta función devuelve una **cadena con los caracteres** introducidos por el usuario.
- Cuando se llama a esta función, el programa se para y espera hasta que el usuario teclee algo. Cuando se presiona **`Return`** o **`Enter`**, el programa continúa e **`input`** retorna lo que ha tecleado el usuario como cadena de caracteres (**`string`**).

```
>>> entrada = input()
Hola 


>>> print(entrada)
Hola
```

Uso de entrada/salida por consola

- Antes de pedir un dato por teclado al usuario, es una buena idea sacar un mensaje informando de lo que se espera.

```
>>> entrada = input("Introduce tu nombre: ")
Introduce tu nombre:
```

- Por este motivo, **`input`** puede llevar como parámetro una cadena de caracteres que se saca por la pantalla justo antes de pedir la entrada al usuario (indicador o **`prompt`** en inglés).

```
>>> entrada = input("Introduce tu nombre: ")
Introduce tu nombre: Jorge Javier
>>> print (entrada)
Jorge Javier
```

- Si necesitamos un número como entrada, un **entero** o un **flotante**, en lugar de una cadena, podemos utilizar las funciones **`int`** y **`float`** para convertir la cadena al tipo numérico correspondiente.

```
>>> base = float(input("Introduce la base de un
triángulo:\n"))
Introduce la base de un triángulo:
7
```

- Aunque hay que tener en cuenta que si lo que introduce el usuario no es un número válido se genera un error al no poder realizar la conversión.

```
>>> base = float(input("Introduce la base de un
triángulo:\n"))
Introduce la base de un triángulo:
Abc

Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: invalid literal for float(): 'Abc'
```

Uso de entrada/salida por consola

Salida por pantalla

- La forma más sencilla de mostrar algo en la salida estándar es mediante el uso de la función **`print`**, como hemos visto en varios ejemplos anteriores. La sintaxis simplificada de la función **`print`** es

```
print(val1, val2,..., sep=" ", end="\n")
```

- donde los argumentos **`sep`** y **`end`** son opcionales (ya que tienen valores por defecto).
- En su forma más básica, a la palabra **`print`** le sigue una cadena de caracteres entre paréntesis. Esta cadena se mostrará en la pantalla del ordenador al ejecutarse la sentencia. Si se omite la cadena de caracteres se produce un salto de línea.

```
>>>A = 30
>>>print("Te costará entre",A,"y", 49.5,"euros")
Te costará entre 30 y 49.5 euros
>>>print(3, 23, 48, sep=" < ")
3 < 23 < 48
>>> print(192, 168, 178, 42, sep=".")
192.168.178.42
```

- Normalmente la función **`print`** muestra algo por la pantalla y se posiciona en la línea siguiente.
- Esto es debido a que el argumento **`end`** por defecto es igual a **`\n`**, que es el carácter especial de salto de línea. Si queremos modificar este comportamiento sólo tenemos que cambiar el valor de **`end`**.

Uso de entrada/salida por consola

- Veamos un ejemplo sobre el programa que calcula el área de un triángulo:

```
#Tema2_2
base = float(input("Dame la base:"))
altura = float(input("Dame la altura:"))
area = (base*altura)/2
print("Base =", base,end=" ")
print("Altura =", altura,end=" ; ")
print("Area =", area)
Dame la base:4
Dame la altura:5
Base = 4.0 Altura = 5.0 ; Area = 10.0
```

- Las cadenas de caracteres que muestra la función **`print`** pueden contener caracteres especiales.
- Se trata de caracteres que van precedidos por la barra invertida **** y que tienen un significado específico. Los más utilizados son **\n**, el carácter de nueva línea, y **\t**, el de tabulación.

```
>>>print("Hola\n\n\tmundo")
Hola

        mundo
```

- Python nos proporciona una alternativa llamada **cadenas formateadas**.
- Una **cadena formateada** es una plantilla en la que las palabras o espacios que permanecen constantes se combinan con marcadores de posición para las variables que se insertarán en la cadena.
- El operador **%** es un operador de cadena llamado **operador de formato.** El lado izquierdo de la expresión contiene la plantilla o cadena de formato y el lado derecho contiene una colección de valores que se sustituirá en la cadena de formato.

```
print(unNombre, "tiene", edad, "años de edad.")
```

- Y mucho más legible así:

```
print("%s tiene %d años de edad.” % (unNombre,edad))
```

Uso de entrada/salida por consola

Carácter	Formato de salida
d, i	Entero
u	Entero sin signo
f	Punto flotante como m.ddddd
e	Punto flotante como m.dddddde+/-xx
E	Punto flotante como m.ddddE+/-xx
g	Usa %e para exponentes menores que −4 o mayores que +5, de lo contrario usa %f
c	Carácter único
s	Cadena o cualquier objeto de datos de Python que pueda convertirse a una cadena usando la función str.
%	Inserta un carácter % literal

Modificadores de anchura

Modificador	Ejemplo	Descripción
número	%20d	Pone el valor en una anchura de campo de 20
-	%-20d	Pone el valor en un campo de 20 caracteres de ancho, justificado a la izquierda
+	%+20d	Pone el valor en un campo de 20 caracteres de ancho, justificado a la derecha
0	%020d	Pone el valor en un campo de 20 caracteres de ancho, rellenando con ceros a la izquierda
.	%20.2f	Pone el valor en un campo de 20 caracteres de ancho con 2 caracteres a la derecha del punto decimal
(nombre)	%(nombre)d	Obtiene el valor del diccionario suministrado usando nombre como clave

Legibilidad de los programas

- Los programas deben ser legibles y conviene que juguemos con un elemento muy básico: el uso de comentarios.

```
#Esto es un comentario
```

- Es buen estilo separar las diferentes zonas del programa con líneas en blanco.
- En el siguiente programa hay tres regiones: la importación de elementos de la librería matemática, los cálculos y la presentación del resultado

```
from math import pi
R=float(input())
P=2*pi*R
print(P)
```

Uso de entrada/salida por consola

- El programa resulta más legible formateado así:

```
from math import pi
radio=float(input("Dame un valor "))
perímetro=2*pi*radio

print(perímetro)
```

- Y mucho más legible así:

```
# Programa para el cálculo del perímetro de una
# circunferencia, cuyo radio se le solicita al
# usuario

# Importamos la constante pi de math
from math import pi

# Solicitamos el valor del radio
radio=float(input("Dame el valor del radio"))

perímetro=2*pi*radio  # Calculamos el perímetro

print(perímetro)  # Comunicamos el resultado
```

- Los programadores pasan muchas horas leyendo programas escritos por ellos mismos o por otros. Las razones son varias:
  - Puede que tengamos que seguir trabajando en un proyecto que abandonamos hace tiempo.
  - O puede tengamos que buscar errores cometidos hace tiempo para corregirlos.
  - O que debamos añadir funcionalidades o adaptarlo a nuevos estándares.
  - O mejorar un programa que escribió otro programador. Cuando hayas de estudiarlo, le estarás infinitamente agradecido si se preocupó de la legibilidad.
  - O escribir programas para demostrar al profesor que has alcanzado los objetivos de una asignatura de programación…
- En todos estos casos (y en muchos otros), haberse asegurado de facilitar la lectura del código será un elemento con un claro impacto en la productividad.

Uso de entrada/salida por consola

- Comparar este programa

```
#Tema2_3.py
h=float(input("Dame h"))
v=float(input("y v"))
z=h*v
print("resultado",z)
v=h*2+v+v
print("y el otro resultado",v)
```

```
Dame h 5
y v 2.5
resultado  12.5
y el otro resultado  15.0
```

- Con este…

```
# Programa: Tema2_4.py
# Propósito: Calcula el perímetro y el área de un rectángulo a partir de
# su altura y anchura.
# Autor: Efrén Arias
# Fecha: 1/5/2018

# Petición de los datos (en metros)
altura = float(input("Dame la altura del rectángulo (en metros)"))
anchura = float(input("Dame la anchura del rectángulo (en metros)"))

# Cálculo del área y del perímetro
area = altura * anchura
perimetro = 2*altura + 2*anchura

# Impresión de resultados por pantalla
print("El área del rectángulo de anchura ",anchura,
      " y altura ",altura," es ",area)
print("El perímetro del rectángulo de anchura ",anchura,
      " y altura ", altura," es ",perimetro)
```

Ejemplos

```
# Tema2_5. Programa para calcular el Seno(x)*x^3
    from math import *
    print("*****************************")
    print("*        Seno(x)*x^3        *")
    print("*****************************")
    x = float(input("*  Introduzca un número  "))
    x = radians(x)  # x=x*pi/180
    print("El resultado da +str(sin(x)*pow(x,3.0)))

# Tema2_6. Programa para calcular el log(x+y)
    from math import log
    print("*****************************\n")
    print("*        log(x + y)         *\n")
    print("*****************************\n")
    x = float(input("*  Introduzca un número  "))
    y = float(input("*  Introduzca otro número "))
    print("El resultado da " + str(log(x + y)))

# Tema2_7. Calcular el IVA de un artículo
    IVA=0.21
    Precio_Producto=float(input("dame el precio del
                                 articulo sin IVA "))
    precioIVA=Precio_Producto*IVA
    print("El precio del IVA es", precioIVA, "€")
    print("El precio final es",
          (precioIVA+Precio_Producto) ,"€")
```

1. Define las variables x = 4, y=5.6, z = 3, c = 4 + 5j, d = −3 + 2j, b =True, m =“Ejemplo de Python” y realiza lo siguiente:
   a. Calcula x + z,
   b. x^z, yz, resto de la división x/z , $(z - x)y^2$, cd, c/d .
   c. Comprueba el tipo de dato de cada variable.
   d. Calcula y + x/z y la división entera de y/x
   e. Comprueba que numero es mayor (x o z).
   f. Calculo el máximo y mínimo de x, y e z.
   g. Calcula el factorial de x

2. Evalúa las siguientes expresiones:

$$sin^2(\pi),\ cos(4\pi/3),\ e^3 log 5,\ 5\,e^{\sqrt{2\pi}},\ \sqrt{\frac{3\pi}{2}},\ e^{7\pi/3},\ 8\pi^{50},\ \sqrt{40\pi^5 + 6\pi + e^{7\pi}}$$

3. Calcula el valor absoluto de x=-8.3 y de y = −4

4. Calcula el valor más próximo, valor por exceso, valor por defecto y parte entera de x=4.2, y=-3.7 y z=-3.6+4.2j.

Uso de entrada/salida por consola

Formateo elegante de la salida

- Hasta ahora encontramos dos maneras de escribir valores: usando **`str.format()`** y usando cadenas formateadas con la función **`print()`**
- Hay otra manera de dar formato a la salida, y es usar literales de cadena con formato,
  - comenzamos una cadena con f o F antes de la comilla de apertura.
  - Dentro de esta cadena, se puede escribir una expresión de Python entre los caracteres { y } que pueden hacer referencia a variables o valores literales.

```
year = 2016
event = 'Referendum'
print(f'Results of the {year} {event}’)
```

- Los literales de cadena con formato (también llamados f-strings para abreviar) permiten incluir el valor de las expresiones de Python dentro de una cadena prefijando la cadena con f o F y escribiendo expresiones como {expresion}.
- La expresión puede ir seguida de un especificador de formato opcional . Esto permite un mayor control sobre cómo se formatea el valor. En el ejemplo siguiente se redondea pi a tres lugares después del decimal:

```
import math
print(f'The value of pi is approximately
{math.pi:.3f}.’)
```

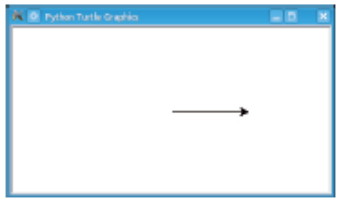
- Pasar un entero después de ':' hará que ese campo sea un número mínimo de caracteres de ancho. Esto es útil para hacer que las columnas se alineen.

Gráficos de Tortuga

Gráficos de tortuga

- Python trae de serie una librería para la implementación de interfaces gráficas de usuario, (crear ventanas, poblarlas con menús, botones, cajas de texto, …) y definir el comportamiento de estos elementos al interactuar con el usuario. La librería se llama **Tkinter**
- Imagina una tortuga que lleva un lápiz en la boca y está esperando nuestras órdenes para dibujar sobre un gran papel extendido en el suelo. Le podemos dar órdenes sencillas (`turtle` usa **Tkinter**)

```
#Tortuga2_1
from turtle import Screen, Turtle
# Creamos una pantalla
pantalla = Screen()
# Dimensionamos la pantalla
pantalla.setup(425, 225)
pantalla.screensize(400, 200)
```



```
# Creamos una tortuga
tortuga = Turtle()
# Hacemos que avance 100 pasos
tortuga.forward(100)

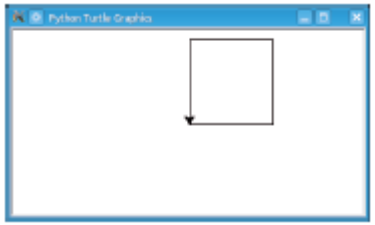
# Esperamos hasta que el usuario cierra la ventana
pantalla.exitonclick()
```

```
#Tortuga2_2
from turtle import Screen, Turtle

# Creamos una pantalla
pantalla = Screen()
# Dimensionamos la pantalla
pantalla.setup(425, 225)
pantalla.screensize(400, 200)

# Creamos una tortuga
tortuga = Turtle()
tortuga.forward(100)
tortuga.left(90)
tortuga.forward(100)
tortuga.left(90)
tortuga.forward(100)
tortuga.left(90)
tortuga.forward(100)

# Esperamos hasta que el usuario cierra la ventana
pantalla.exitonclick()
```



Gráficos de Tortuga

- Aquí relacionamos los métodos básicos de dibujo con la tortuga.

backward(d)	Retrocede d pasos.
begin_fill()	Comienza a dibujar una figura a rellenar.
circle(r)	Dibuja un círculo de radio r. El círculo está centrado a r pasos a la izquierda de la tortuga.
clear()	Borra la pantalla
color(t1,t2)	Cambia los colores de dibujo y fondo.
dot(d)	Dibuja un punto de diámetro d centrado en la posición actual.
done()	Finaliza el uso de la Tortuga
end_fill()	Termina de dibujar una figura a rellenar.
forward(d)	Avanza d pasos.
goto(x, y)	Ubica a la tortuga en la posición de coordenadas (x,y).
home()	Ubica a la tortuga en la posición de coordenadas (0,0).
left (g)	Gira a la izquierda g grados.
pencolor(c)	Usa el color c para los trazos, donde c es una cadena con el nombre del color en inglés. ('white', 'black', 'red', 'blue', 'green', 'cyan', 'magenta', 'yellow', 'pink' y 'orange')
pendown(), down()	Baja el lápiz.

pensize(s)	Usa un lápiz con trazo de s píxeles de grosor.
penup(), up()	Levanta el lápiz.
reset()	Borra la pantalla y recoloca la tortuga.
right(g)	Gira a la derecha g grados.
setheading(g)	Hace que la tortuga apunte en dirección g grados (donde 0 grados es la dirección este).
setworldcoordinates(x1,y1,x2,y2)	Establece el sistema de coordenadas de la pantalla, donde (x1,y1) representa el vértice inferior izquierdo y (x2,y2) el vértice superior derecho.
setx(x)	Ubica a la tortuga en la posición de abscisa x y la misma ordenada actual.
sety(y)	Ubica a la tortuga en la posición de ordenada y y la misma abcisa actual.
speed(v)	Permite controlar la velocidad con la que se desplaza la tortuga su argumento v es un número entre 0 y 10
towards(x,y)	Hace que la tortuga apunte en dirección al punto (x,y).
Turtle(forma)	donde forma puede ser 'arrow', 'blank', 'circle', classic', 'square', 'triangle' o 'turtle'
width(i)	Dibuja lineas de grosor i
write(t)	Escribe el texto de la cadena t en pantalla, en la posición actual de la tortuga.

Gráficos de Tortuga

```
#Tortuga2_4
from turtle import Screen, Turtle

# Creamos una pantalla
pantalla = Screen()
# Dimensionamos la pantalla
pantalla.setup(425, 425)
pantalla.screensize(400, 400)
pantalla.setworldcoordinates
   (-50, -150, 350, 250)

# Creamos una tortuga
tortuga = Turtle('turtle')

tortuga.pensize(3)
tortuga.dot(10)
tortuga.forward(100)
tortuga.dot(10)
tortuga.forward(100)
tortuga.dot(10)
tortuga.forward(100)
tortuga.dot(10)
```

```
tortuga.penup()
tortuga.goto(0, 100)
tortuga.pendown()

tortuga.pencolor('red')
tortuga.pensize(5)
tortuga.circle(20)
tortuga.forward(50)
tortuga.pensize(4)
tortuga.left(20)
tortuga.circle(20)
tortuga.forward(50)
tortuga.pensize(3)
tortuga.left(20)
tortuga.circle(20)
tortuga.forward(50)
tortuga.pensize(2)
tortuga.left(20)
tortuga.circle(20)
tortuga.forward(50)
tortuga.pensize(1)
tortuga.left(20)
tortuga.circle(20)
```

```
tortuga.forward(50)

tortuga.penup()
tortuga.goto(0, -100)
tortuga.towards(0, 0)

tortuga.write('Hola')
tortuga.backward(20)
tortuga.write('Adios')
# Esperamos hasta que el usuario
# cierra la ventana
pantalla.exitonclick()
```

